Discovery Report

Handling Sales Day Demand for the Onward Logistics Order System

Prepared by: Patrick Ndaziramiye

Date: 9 November 2026

Module: Compute & Delivery on AWS, Week 3

AWS Region: eu-west-1 (Ireland) | Delivery: CloudFormation with Git sync

Executive Summary

On sales days, order traffic at Onward Logistics reaches about three times a normal day, at times nobody can schedule. Today's fixed fleet either sits mostly idle or slows down at the peak. We recommend one public Application Load Balancer in front of an EC2 Auto Scaling group in private subnets across two Availability Zones. A CPU target-tracking policy adds servers during the rush and removes them afterwards.

A Regional NAT Gateway gives the servers outbound-only access to updates. Admin access goes through AWS Systems Manager, so no admin ports are open. The whole environment is defined in CloudFormation and deployed from Git through Git sync, so it can be rebuilt from the repository at any time.

Key figures

- Sales day traffic: about 3 times normal, at unpredictable times.
- Servers: 2 to 6, scaled automatically on CPU (target 50%).
- Availability: 2 Availability Zones; survives the loss of a server or a whole data centre.
- Exposure: 1 public entry point; every server is private.
- Cost: about $3.75 per quiet day; a sales day adds about $1.60.

Part A: What We Found

A1. The Problem

Onward Logistics runs its order system on a fixed set of servers. On sales days demand jumps to roughly three times normal, and nobody can predict exactly when it will start.

This leaves two poor options. Sizing for the peak means paying for idle servers about 95% of the month. Sizing for a normal day means the system slows down or fails exactly when merchants need it most. In addition, the failure of one server or one Availability Zone can take orders offline.

A2. Common Solutions Compared

Approach	How it works	Fit for Onward Logistics
Over-provision for peak	Run peak capacity all the time	Rejected: pays for about 3 times the capacity most of the month
Manual scaling	Operators add servers when they see load	Rejected: slow, error-prone, needs someone on call
Scheduled scaling	Add capacity at fixed times	Complement only: sale timing is unpredictable
Step scaling	Alarms with hand-tuned scaling steps	Not needed: more to tune than target tracking

Approach	How it works	Fit for Onward Logistics
Target-tracking auto scaling	Keep average CPU at a target; scale out and in automatically	**Chosen:** hands-off, little tuning, cost follows demand
Serverless (Lambda / Fargate)	The platform scales per request or task	Future option: the brief requires EC2 behind a load balancer
Managed PaaS (Elastic Beanstalk)	Platform-managed fleet	Future option: hides network controls the client wants
CDN / caching in front	Serve static content from the edge	Complement only: does not help order writes

A3. Impact of Doing Nothing

- **Lost orders and merchants:** slow pages and errors at the peak turn away customers exactly when demand and revenue are highest.
- **Damaged trust:** merchants who rely on Onward on their busiest day move to competitors after one bad sales day.
- **Night-time firefighting:** engineers add servers by hand under pressure, with a high risk of mistakes and burnout.
- **Paying for idle capacity:** the only "safe" alternative is a peak-sized fleet all month, about 3 times the compute bill.
- **Single point of failure:** a single server or Availability Zone outage takes the order system offline.
- **Exposed servers:** publicly reachable servers and open admin ports widen the attack surface.

A4. Security and Resilience Patterns

- **Public ALB, private compute:** only the load balancer has a public address. Servers sit in private subnets with no public IPs.
- **Security group chaining:** the app port accepts traffic only from the ALB's security group, with no CIDR rules and no port 22.
- **Outbound-only internet:** a Regional NAT Gateway lets servers fetch updates but blocks all inbound connections.
- **Multi-AZ self-healing:** two Availability Zones, a minimum of 2 servers and ELB health checks; failed servers are replaced automatically.
- **No admin ports:** SSM Session Manager and Run Command over HTTPS replace SSH and bastion hosts.
- **Secrets and least privilege:** the token is kept in Secrets Manager, IAM uses managed policies only, and IMDSv2 is enforced.

A5. Sources

- Target tracking scaling policies for Amazon EC2 Auto Scaling
- Elastic Load Balancing health checks for an Auto Scaling group
- What is an Application Load Balancer?
- Regional NAT gateways for automatic multi-AZ expansion
- AWS CloudFormation Git sync
- AWS Systems Manager Session Manager
- AWS Well-Architected Framework: Reliability, Security and Cost Optimization pillars

Target Architecture

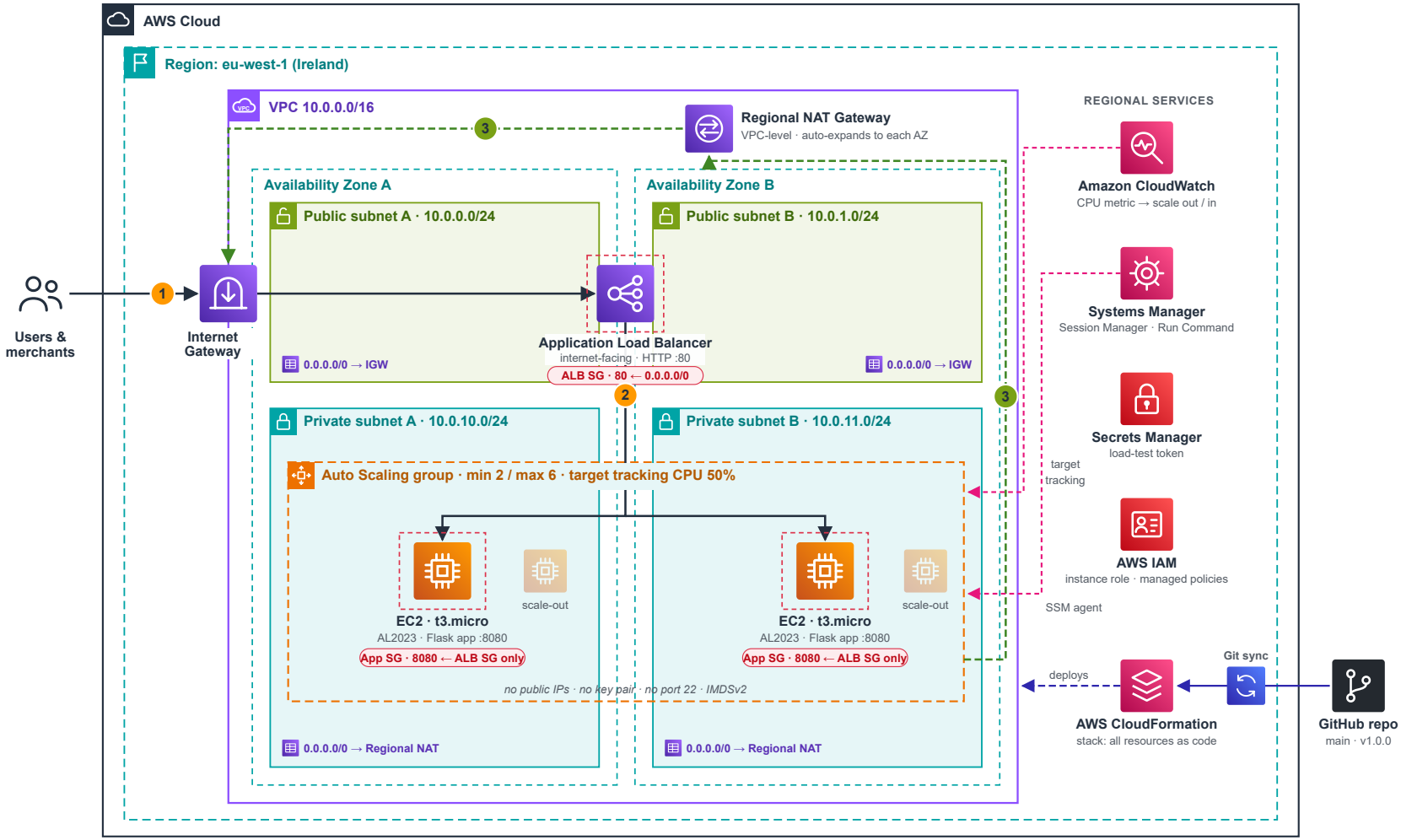


Figure 1. Onward Logistics order system on AWS

Traffic flows: (1) Users reach the Application Load Balancer through the Internet Gateway; the ALB is the only public endpoint. (2) The ALB forwards requests to EC2 instances on port 8080 in both Availability Zones and checks `GET /health` every 15 seconds. (3) EC2 instances reach the internet outbound only, through the Regional NAT Gateway, for updates, git, pip and SSM.

Part B: Business Value

B1. Chosen Approach

A single internet-facing Application Load Balancer is the only public entry point. Behind it, an EC2 Auto Scaling group of small `t3.micro` servers runs in private subnets across two Availability Zones. A target-tracking policy keeps average CPU near 50%, adding servers (up to 6) during a sales rush and removing them (down to 2) afterwards. Nobody has to touch it.

Each new server installs the order application itself at boot from a pinned Git release, and each page shows which server and Availability Zone answered. A Regional NAT Gateway lets the servers download updates without being reachable from the internet. Everything is defined in one CloudFormation template that deploys automatically from Git through Git sync, so the environment can be rebuilt from the repository.

- **Scale out:** CPU above 50% triggers a CloudWatch alarm and the Auto Scaling group launches servers (maximum 6).
- **Scale in:** after about 15 minutes of low CPU the group removes extra servers (minimum 2).
- **Self-heal:** a failed `/health` check makes the ALB mark the server unhealthy, and the group replaces it.

B2. Services and Why

AWS service	Role in the solution	Why this choice
Amazon VPC	Public and private subnets in 2 AZs, route tables	Network isolation and AZ fault tolerance
Application Load Balancer	Single public address, spreads requests, health checks	Stable endpoint while servers come and go
EC2 Auto Scaling	Keeps 2 to 6 healthy servers, target tracking at 50% CPU	Elastic and self-healing, scales both ways
Launch Template and EC2	AL2023, t3.micro, IMDSv2, UserData installs the app	Every server built identically, Free Tier sized
Amazon CloudWatch	CPU metric and alarms behind target tracking	Managed by the policy, nothing to hand-tune
Regional NAT Gateway	Outbound-only internet for all AZs	One resource, expands into each AZ automatically
AWS Systems Manager	Session Manager shell, Run Command load test	Admin access with zero open ports
AWS Secrets Manager	Generated token for the load-test endpoint	Secret never stored in code or Git
AWS IAM	Instance role with managed policies only	Least privilege, auditable, no inline policies
CloudFormation and Git sync	Whole stack as code, deployed on every push	Repeatable, reviewable, rebuildable from Git

B3. Who Benefits

- **CTO (elasticity):** capacity follows demand automatically, in both directions, with no one touching it.